智驾伴侣项目知识点介绍

从业务到微服务技术栈

这份材料把 PPT 中会出现的概念按顺序解释清楚。阅读时先抓住一句话结论，再看项目里的具体用法；不需要一次记住所有英文，答辩时能说明每个组件解决什么问题即可。

汇报人　周轩亦

学号　2435050112

日期　2026 年 11 月

先看整体 项目在做什么

智驾伴侣是一个面向车主的车辆数据服务平台。车主把车辆和行程数据录入系统后，平台可以保存告警、分析驾驶行为、生成月度报告，并根据油耗与电耗计算碳排放。项目当前的重点是后端微服务，而不是完整的前端产品。

PPT 中反复出现的 vehicleId 是业务关联键。它表示“这辆车的编号”，让不同服务知道数据属于哪辆车，但各服务不会直接读取对方数据库。

服务	端口	当前作用	数据库
user-service	8081	车主基础接口，目前是简化实现	user_db
vehicle-service	8082	创建和查询车辆档案	vehicle_db
alert-service	8083	创建告警并校验车辆	alert_db
driving-behavior-service	8084	保存驾驶事件并生成月报	driving_db
carbon-service	8085	读取驾驶月报并计算碳排	carbon_db

最重要的结论：五个服务通过 HTTP 协作，但数据仍归各自服务所有。这样修改一个服务时，不需要直接改另一个服务的数据库。

单体应用与微服务

单体应用是把车辆、告警、驾驶员等模块写在一个项目中，最后打包成一个程序。它的优点是启动和部署简单，模块之间调用也直接；缺点是所有代码一起发布，一个模块出问题可能影响整个系统，团队并行开发也容易互相牵连。

微服务是把一个系统按业务能力拆成多个可以独立运行的小程序。智驾伴侣把车辆、告警、驾驶行为和碳排分别放进不同服务。拆分以后，每个服务有自己的程序、端口、数据库和部署单元。

比较项	单体应用	微服务
部署单位	一个整体程序	多个独立程序
数据库	通常集中或共享	每个服务独立数据库
修改影响	可能牵动整个项目	影响面通常限制在服务内
运行复杂度	较低	需要注册发现、部署编排和容错

微服务并没有自动消除复杂度。它把代码之间的复杂度，转移成了服务通信、部署和故障处理的复杂度，所以项目还需要 Nacos、Docker Compose、超时和降级处理。

服务边界与数据库归属

服务边界可以理解为一句话：“哪类数据应该由哪个服务负责。”车辆档案归车辆服务，告警记录归告警服务，驾驶事件归驾驶服务。一个服务只通过自己的 Repository 访问自己的数据库。

项目采用了“每个服务一个数据库”的做法。alert_db 里只保存 vehicleId，不建立指向 vehicle_db 的外键。这种做法叫逻辑关联，它只能说明业务上有关联，数据库本身并不知道另一张表在哪里。

概念	这句话可以这样理解
独立数据库	每个服务管理自己的表，不直接操作别人的库
业务 ID	保存 vehicleId，用编号表示关联对象
跨库查询	一个服务直接查另一个服务的数据库，本项目禁止
Repository	Spring Data JPA 中负责数据库读写的接口

这样做的好处是边界清楚，代价是需要通过 HTTP 请求获取其他服务的信息，还要处理对方不可用的情况。

Nacos 服务注册与发现

容器启动后，IP 地址可能变化，因此调用方不能把 vehicle-service 的地址写死在代码里。

Nacos 是注册中心，它像一个动态通讯录：服务启动后把自己的名称、地址和端口登记进去，调用方需要时再按服务名查询。

登记动作叫服务注册，按名称查找实例叫服务发现。COURSE_GROUP 是 Nacos 中的分组，用来把本课程相关服务放在一起。一个服务可能有多个实例，每个实例就是一份可以接收请求的运行程序。

概念	在项目中的值	用途
注册中心	Nacos 3.2.4	保存服务实例的动态地址
分组	COURSE_GROUP	隔离本课程的服务
控制台端口	18080	浏览器查看服务列表
HTTP 服务端口	8848	Nacos 服务端 HTTP API
SDK gRPC 端口	9848	业务服务 SDK 使用

服务进程会持续向 Nacos 发送心跳。实例停止后，Nacos 在一定时间内将它摘除；恢复运行后，实例重新注册。verify-discovery 脚本验证的就是“找到服务、服务下线、服务恢复”这一过程。

HTTP API 与状态码

API 是程序对外提供的调用入口。HTTP API 通常由请求方法、路径和请求内容组成。GET 表示查询，POST 表示新增。例如 GET /api/vehicles/1 表示查询编号为 1 的车辆，POST /api/alerts 表示创建一条告警。

JSON 是一种常用的数据格式，用大括号和字段名表示结构化数据。例如 {"vehicleId":1,"level":2} 表示车辆编号为 1、告警等级为 2。控制器接收到 JSON 后，把它转换成 Java 对象。

状态码	含义	项目中的例子
200	请求成功	查询或创建成功后返回
201	资源已创建	创建驾驶事件和碳排报告
400	请求内容不合法	告警等级不合法或车辆不存在
404	找不到资源	查询指定车辆或月度报告失败
503	依赖服务当前不可用	依赖服务没有可用实例

状态码不是随便选的。它让调用方不需要读自然语言，也能判断下一步应该重试、修改请求，还是把错误返回给用户。

Spring Boot 的分层结构

Spring Boot 是 Java 的企业应用开发框架。它提供 Web、数据库、配置和监控等常用能力，让开发者可以把精力放在业务代码上。项目中的每个服务都是一个 Spring Boot 应用。

一个典型请求会经过 Controller、Service、Repository 和数据库。Controller 负责接收 HTTP 请求；Service 负责业务规则；Repository 负责数据读写；Entity 表示数据库表的 Java 对象；DTO 表示服务之间传递的数据结构。

代码层	解决的问题	项目例子
Controller	接收和返回 HTTP 数据	AlertController 创建告警
Service	执行校验和业务计算	AlertService 校验等级和车辆
Repository	访问数据库	AlertRepository 保存告警
Entity	映射数据库记录	Vehicle、Alert、DrivingEvent
DTO	定义请求和响应格式	AlertRequest、CarbonReportRequest

JPA 和 Hibernate 负责把 Java 对象与数据库表对应起来。测试时项目使用 H2 内存数据库，因此测试启动快，也不会修改真实 MySQL 数据。

跨服务调用与容错

RestTemplate 是 Spring 提供的 HTTP 客户端，可以直接写出请求地址并获取响应。DiscoveryClient 先向 Nacos 查询实例地址，再把地址交给 RestTemplate。驾驶行为服务校验车辆、碳排服务读取月报时使用这种方式。

OpenFeign 把远程 HTTP 调用声明成 Java 接口。开发者写一个带 @FeignClient 的接口，指定服务名和路径，调用时像调用本地方法一样。告警服务当前使用 OpenFeign 调用 vehicle-service。

Fallback 是降级处理。被调用服务不可用时，程序不继续执行正常流程，而是进入预先准备的失败分支。项目的降级逻辑会阻止告警落库，避免车辆尚未确认就产生无效数据。

概念	一句话解释
LoadBalancer	服务有多个实例时，选择一个实例发送请求
Fallback	远程调用失败时执行备用处理
Circuit Breaker	失败过多时暂时停止继续请求，防止故障扩散
降级	牺牲部分功能或立即报错，保证主流程不被拖垮

告警业务规则

创建告警时，告警服务先检查 level 是否在 1 到 3 之间，再调用车辆服务确认 vehicleId 对应的车辆存在。两项都通过后，系统把状态设为 pending，最后写入 alert_db。

如果等级不合法，接口返回 400。车辆不存在也返回 400。车辆服务没有可用实例时返回 503。这个顺序说明业务校验发生在写库之前，失败请求不会留下半成品记录。

“脏数据”在这里指没有合法业务依据、却已经被保存的数据。例如车辆不存在，告警记录却已经写入，就会让后续查询和处理变得不可信。

当前代码中 alert-service 已经切换到 OpenFeign，驾驶行为和碳排服务仍使用 DiscoveryClient 加 RestTemplate。两者目标相同，都是通过 Nacos 找到服务后再发 HTTP 请求。

驾驶月报与碳排计算

驾驶行为服务按车辆和月份查询驾驶事件，再汇总里程、油耗、电耗、急加速次数、急刹车次数和高速事件次数。若这个月没有任何驾驶数据，就返回资源不存在。

月度评分从 100 分开始，急加速每次扣 2 分，最多扣 30 分；急刹车每次扣 3 分，最多扣 30 分；最高速度超过 120 km/h 的事件每次扣 2 分，最多扣 20 分。公式可以写成：

score = max(0,

100 - min(30, 急加速次数 × 2)

- min(30, 急刹车次数 × 3)

- min(20, 高速事件数 × 2))

碳排服务先通过 HTTP 读取驾驶月报，再用默认系数计算二氧化碳排放：每升汽油按 2.31 千克 CO_2，每千瓦时电力按 0.581 千克 CO_2。

CO_2 千克数 = 汽油升数 × 2.31 + 用电千瓦时数 × 0.581

同一车辆、同一月份重复生成报告时，系统更新原报告，不重复插入。这个唯一关系写在 vehicleId + report_month 上。

Docker 部署与自动化验证

镜像是程序及其运行环境的打包结果，容器是镜像运行后的进程。Dockerfile 定义如何构建镜像，Docker Compose 则用一个 YAML 文件启动多个容器，并配置端口、环境变量、依赖关系和数据卷。

概念	初学者理解
Image 镜像	程序的安装包和运行环境模板
Container 容器	根据镜像启动的一次运行实例
Volume 数据卷	把数据库数据保存在容器之外
Port Mapping 端口映射	把宿主机端口转到容器端口
Environment Variable 环境变量	不写死在代码里的配置，例如密码和服务地址
Health Check 健康检查	判断容器是否已经可以正常提供服务

测试使用 H2 内存数据库和 Mock 对象，验证单个服务的业务逻辑，不依赖真实 MySQL 或 Nacos。冒烟测试检查系统能否启动并响应，服务发现脚本进一步检查正常、下线和恢复三个场景。

尚未完成的能力与后续路线

当前项目已经完成基础服务拆分和调用验证，但还没有面向用户的前端，也没有 BFF 聚合层。BFF 是专门为某类前端整合多个后端接口的一层服务，可以减少前端分别调用多个微服务的负担。

API 网关位于所有客户端和后端服务之间，负责统一入口、路由和鉴权。Nacos Config 负责集中管理配置，并支持配置变化后刷新。SkyWalking、Prometheus 和 Grafana 用于观察请求链路、指标和运行状态。

消息队列适合把不强依赖立即完成的工作异步处理。Seata 用于分布式事务，但它不能代替清晰的业务设计。当前项目先用校验、状态码和失败阻断保证基础一致性。

老师可能追问	简短回答
为什么拆分	按业务能力独立开发、部署和数据管理
为什么使用 Nacos	服务地址动态变化，调用方按服务名查找实例
为什么不能跨库查询	会破坏服务边界，使数据库重新耦合
怎样避免脏数据	写库前校验，依赖失败时 fallback 或返回 503
怎样证明能运行	11 个测试、冒烟测试和上下线恢复脚本

答辩时可以把这套项目概括为：五个服务按业务边界拆分，各自管理数据，通过 Nacos 发现彼此，用 HTTP 完成必要协作，并通过测试和脚本验证主流程。